Azimi Innovation Lab
AI Architect · End-to-End Projects

جلسه ۳

توابع، ماژول‌ها و سازماندهی کد

چطور کدهای بزرگ‌تر را تمیز، قابل فهم و قابل استفاده مجدد کنیم

ادامه مستقیم جلسات ۱ و ۲ — Azimi Innovation Lab

Azimi Innovation Lab
AI Architect · End-to-End Projects

اتصال به جلسات قبل: در جلسه ۱ یاد گرفتید کد را بخوانید و تست کنید. در جلسه ۲ متغیرها، انواع داده و منطق را فهمیدید. حالا وقت آن است که ببینید چطور این قطعات را داخل توابع قرار می‌دهیم تا کد تکراری نشود و قابل مدیریت بماند.

۱. تابع چیست و چرا ضروری است؟

تابع یک قطعه کد است که یک کار مشخص انجام می‌دهد و می‌توانیم آن را بارها صدا بزنیم. به جای اینکه یک کار را چند بار بنویسیم، آن را یک بار داخل تابع می‌گذاریم و هر جا لازم شد صدا می‌زنیم.

مثال از زندگی: دستور پخت کیک را یک بار می‌نویسید. هر بار که می‌خواهید کیک بپزید، همان دستور را دنبال می‌کنید؛ نیازی نیست دوباره از اول بنویسید. تابع در کد همین نقش را دارد.

```
function calculateTotal(price, quantity) {
    return price * quantity;
```

```
}

let total1 = calculateTotal(150000, 2);  // 300000
let total2 = calculateTotal(80000, 3);   // 240000
```

تابع calculateTotal دو ورودی می‌گیرد و نتیجه را برمی‌گرداند. بدون تابع مجبور بودیم ضرب را هر بار تکرار کنیم.

۲. اجزای اصلی یک تابع

- نام تابع: باید واضح باشد (calculateTotal بهتر از calc یا f است).
- پارامترها (ورودی): مقادیری که تابع دریافت می‌کند.
- بدنه تابع: کاری که انجام می‌دهد.
- **return** (خروجی): مقداری که تابع برمی‌گرداند (گاهی تابع فقط کاری انجام می‌دهد و چیزی برنمی‌گرداند).

```
function greetUser(name) {
    if (!name) {
        return "نام وارد نشده";
    }
    return "سلام " + name + "! خوش آمدید";
}
```

```
console.log(greetUser("سارا"));     // سلام سارا! خوش آمدید
console.log(greetUser(""));         // نام وارد نشده
```

۳. توابع در کدهای تولیدشده توسط AI

هوش مصنوعی معمولاً کد را به توابع تقسیم می‌کند. وقتی فایلی را باز می‌کنید، دنبال این الگوها بگردید:

- توابعی که با function یا const ... = () => شروع می‌شوند.
- توابعی که رویدادها را مدیریت می‌کنند (مثل handleClick یا onSubmit).
- توابعی که داده را بررسی یا تبدیل می‌کنند (validate، format، calculate).

نکته کلیدی برای خواندن: وقتی به یک تابع رسیدید، اول بپرسید «این تابع چه ورودی می‌گیرد و چه خروجی می‌دهد؟» سپس بدنه را بخوانید. نیازی نیست همه جزئیات را یک‌جا بفهمید.

۴. مثال واقعی: تابع اعتبارسنجی فرم

در پروژه‌های شما اغلب فرمی وجود دارد. AI ممکن است تابعی شبیه این بنویسد:

```
function validateRegistration(user) {
    const errors = [];

    if (!user.name || user.name.length < 2) {
        errors.push("نام باید حداقل ۲ حرف باشد");
    }
    if (!user.email || !user.email.includes("@")) {
        errors.push("ایمیل معتبر نیست");
    }
    if (user.age < 18) {
        errors.push("سن باید حداقل ۱۸ باشد");
    }

    return {
        isValid: errors.length === 0,
        errors: errors
    };
}
```

این تابع یک شیء user می‌گیرد، چند شرط را بررسی می‌کند و یک شیء حاوی وضعیت و لیست خطاها برمی‌گرداند. اگر این ساختار را بفهمید، می‌توانید بفهمید چرا فرم خطا می‌دهد و چطور آن را اصلاح کنید.

۵. سازماندهی کد و ماژول‌ها

وقتی پروژه بزرگ می‌شود، همه کد را در یک فایل نمی‌گذاریم. آن را به فایل‌های جدا (ماژول) تقسیم می‌کنیم:

- یک فایل برای توابع مربوط به کاربر (user.js)
- یک فایل برای توابع مربوط به فرم (form.js)
- یک فایل برای نمایش صفحه (ui.js)

در جاوااسکریپت معمولاً با import و export این فایل‌ها به هم وصل می‌شوند. وقتی AI چند فایل به شما داد، دنبال کلمه‌های import و export بگردید تا بفهمید کدام فایل از کدام استفاده می‌کند.

```
// فایل utils.js
export function formatPrice(price) {
    return price.toLocaleString() + " تومان";
}

// فایل اصلی
import { formatPrice } from "./utils.js";
console.log(formatPrice(150000));  // 150,000 تومان
```

۶. اشتباهات رایج AI در توابع

- تابعی را صدا می‌زند اما تعریفش را در فایل دیگری گذاشته و فراموش کرده آن را import کند.
- پارامترها را اشتباه نام‌گذاری می‌کند یا تعدادشان با صدا زدن مطابقت ندارد.
- تابع خیلی طولانی می‌نویسد (بیش از ۲۰–۳۰ خط) که خواندنش سخت می‌شود.
- خروجی تابع را مشخص نمی‌کند یا گاهی return می‌کند و گاهی نه.

راه حل: تابع را جداگانه تست کنید. مقادیر مختلف به آن بدهید و ببینید خروجی درست است یا نه.

۷. تمرین‌های عملی

تمرین ۱: خواندن تابع

از AI بخواهید یک تابع بنویسد که قیمت و تخفیف را بگیرد و قیمت نهایی را برگرداند. کد را بخوانید و به این سوالات جواب دهید:

- تابع چند پارامتر دارد؟
- چه زمانی تخفیف اعمال می‌شود؟
- اگر تخفیف منفی باشد چه اتفاقی می‌افتد؟

تمرین ۲: شکستن کد طولانی

از AI بخواهید یک کد نسبتاً طولانی (مثلاً مدیریت لیست کارها) بنویسد. سپس از آن بخواهید همان کد را به ۳–۴ تابع کوچک‌تر تقسیم کند. دو نسخه را مقایسه کنید و بگویید کدام خواناتر است.

تمرین ۳: تست تابع

تابع اعتبارسنجی بالا را کپی کنید. سه شیء مختلف به آن بدهید (یکی درست، یکی با ایمیل غلط، یکی با سن کم) و خروجی را ببینید.

۸. چک‌لیست پایان جلسه

- می‌توانم توضیح دهم تابع چیست و چرا از تکرار کد جلوگیری می‌کند.
- اجزای تابع (نام، پارامتر، بدنه، return) را می‌شناسم.
- حداقل یک تابع را خوانده و تست کرده‌ام.
- می‌دانم import و export برای وصل کردن فایل‌ها استفاده می‌شوند.
- تمرین‌ها را انجام داده‌ام.

Azimi Innovation Lab
AI Architect · End-to-End Projects

۹. خلاصه و پل به جلسه بعد

در این جلسه یاد گرفتید که توابع کد را سازماندهی می‌کنند، از تکرار جلوگیری می‌کنند و تست کردن را آسان‌تر می‌سازند. وقتی AI کدی می‌نویسد، اول توابع را پیدا کنید و هر کدام را جداگانه بفهمید.

جلسه بعد: HTML و CSS — فهم عمیق ساختار و ظاهر صفحات وب که در پروژه‌های شما زیاد استفاده می‌شود.

پایان جلسه ۳ — Azimi Innovation Lab

حالا می‌توانید کدهای سازمان‌یافته را بهتر بخوانید و مدیریت کنید.

موفق باشید!